

Day 14: Production edge AI design and capstone start

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 7, 2026

- ▶ On Days 1 to 13 you built the parts of an edge AI system: a model on a board, a stream, messages, decisions, and monitoring. A product has many devices that run for years.
- ▶ Each device needs new models and fixes over a bad network. A failed update must not stop the device. The devices of one fleet often have different hardware.
- ▶ A device in a public place can be stolen, opened, or attacked through its network. Its model, its keys, and its data need protection.
- ▶ Today you also start the capstone project: your team writes a design proposal with requirements, boards, a model, data, metrics, and budgets, and the instructor reviews it.

After this day you can:

- ▶ Plan model versions and over-the-air updates for a group of devices.
- ▶ Identify security and privacy threats of an edge AI system and select mitigations.
- ▶ Estimate the power budget and the cost of a deployment.
- ▶ Write a system design proposal.

Part 1 Life cycle in production

Exercise: write the update plan for 100 devices with a bad network

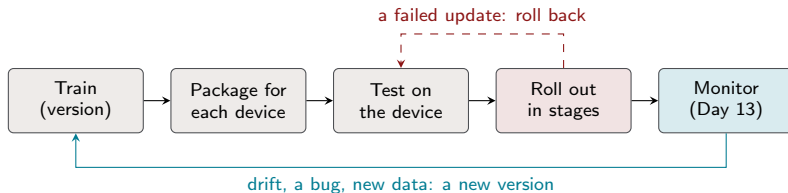
Part 2 Security, privacy, and reliability

Exercise: list the threats for the Day 12 system and select two mitigations

Part 3 Design method

Exercise: review a given system design and find three weak points

A deployed model has a life



- ▶ A device runs for years. Its model and its program change many times in this time, often with no person near the device.
- ▶ Each step needs a name for what it moves: a version.
- ▶ An update over the network is “over the air” (OTA). It must not stop the device, and a failed update must be undone.

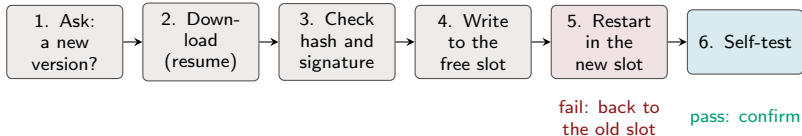
Key point: Plan the second version before you ship the first one.

```
{"model": "cupbottle-detector", "version": "1.4.0",  
  "file": "mnv2_int8.tflite", "bytes": 4107968,  
  "sha256": "0ee20427d55c421fb29734fb17fbad6df2e6f5498359309f04738fe3f6805e8c",  
  "runtime": "ai-edge-litert 2.2.0", "input": "224x224 RGB, ImageNet norm",  
  "data": "train-2026-09", "test_accuracy": 0.752, "min_app": "2.1.0"}
```

- ▶ A version number for each part: the model, the program, the runtime, the settings. A model works only with the program and the pre-processing that it was tested with (Day 13: skew).
- ▶ The hash (SHA-256) proves that the file on the device is the file that you tested. The device checks it before it uses the file.
- ▶ The manifest also names the data, the test result, and the oldest program that can run the model.

Source: Example manifest. The size, the hash, and the accuracy (75.2 %) are those of the Day 7 lab file `mnv2_int8.tflite`.

The steps of an over-the-air update



- ▶ The device keeps the old version until the new one passes its test. Two slots: one runs, the other receives the update (A/B).
- ▶ A device that loses power in any step still starts: the old slot is complete.
- ▶ A device that does not confirm in time (a crash, a stop) goes back to the old slot at the next start.

Name	Type	Size
nvs	settings	20 KB
otadata	which slot runs	8 KB
app0	program slot A	3264 KB
app1	program slot B	3264 KB
spiffs	files	1536 KB
coredump	crash record	64 KB

The default partition table of the board (8 MB of flash) has two program slots of 3 342 336 bytes each.

- ▶ The model is in the program (Day 3: `model.h`), so a new model is a new program. The Day 9 sketch with four models has 944 679 bytes: it fits one slot.
- ▶ The scheme “Maximum APP (No OTA)” gives one slot of 7.9 MB and no update over the air.
- ▶ The Arduino core has the libraries `Update`, `HTTPUpdate`, and `ArduinoOTA`, and its bootloader has the rollback on.

Source: esp32 Arduino core 3.3.12: `tools/partitions/default_8MB.csv`, `boards.txt`, `sdkconfig` of the ESP32-S3.

```
#include "esp_ota_ops.h"

bool selfTest() { return true; }           // Day 3: the stored test window

bool verifyRollbackLater() { return true; } // the sketch decides, not the core

void setup() {
  Serial.begin(115200);
  const esp_partition_t *slot = esp_ota_get_running_partition();
  Serial.printf("running from %s\n", slot->label); // app0 or app1
  if (selfTest()) {
    esp_ota_mark_app_valid_cancel_rollback();      // keep this version
  } else {
    esp_ota_mark_app_invalid_rollback_and_reboot(); // back to the old slot
  }
}
```

- ▶ A new program starts in the state “pending”. With no `verifyRollbackLater()`, the core confirms it at once.
- ▶ The self-test runs the model on a stored input and compares the output, as the test window of Day 3.

Source: Hooks of the esp32 Arduino core 3.3.12 (`esp32-hal-misc.c`). The sketch compiles for XIAO_ESP32S3.

```
models/1.3.0/   models/1.4.0/   models/current -> 1.3.0
sha256sum -c 1.4.0/SHA256SUMS      # check the new files
ln -sfn 1.4.0 current.new          # a new link ...
mv -T current.new current          # ... replaces the old in one step
sudo systemctl restart detector    # the program loads models/current
ln -sfn 1.3.0 current.new && mv -T current.new current      # roll back
```

- ▶ The program always opens `models/current`. A rename is atomic on Linux: the link points to the old or to the new folder, never to a half-written file.
- ▶ Keep the old folder until the new version passed its health check.
- ▶ An update of the system and of the libraries is larger. Tools such as Mender use two root file systems (A/B) for that.

Source: The first line shows the folders, not a command.

Trigger	Signal	Who acts
No start	No confirm after the restart	The bootloader, at once
Self-test fails	Wrong output for the stored input	The program, at once
Crash loop	Restarts in one hour above a limit	A watchdog, minutes
Health check	No frames, latency, memory (Day 13)	The fleet server, minutes
Model metrics	Confidence, class counts against the old version	A person, hours

- ▶ The first rows need no network: the device saves itself.
- ▶ A model metric needs a reference and enough samples (Day 13). With no labels, a change says “different”, not “worse”.
- ▶ A device can run the old model as a shadow next to the new one and compare their outputs on the same frames.

Source: *Machine Learning Systems*, Volume II, chapters “ML Operations at Scale” and “Edge Intelligence”.

$$R_{\text{rollout}} = p_{\text{regression}} \times I_{\text{regression}} \times E_{\text{exposure}}$$

5



- ▶ The risk of a rollout is the chance of a regression, times its impact, times the exposure: the devices that run the new version before you see the problem.
- ▶ A canary group of 5 devices limits the exposure of a bad version to 5 devices, not 100. Each stage waits for a health check.
- ▶ Select the canary devices with care: they must be online and must cover each hardware class and each type of place.

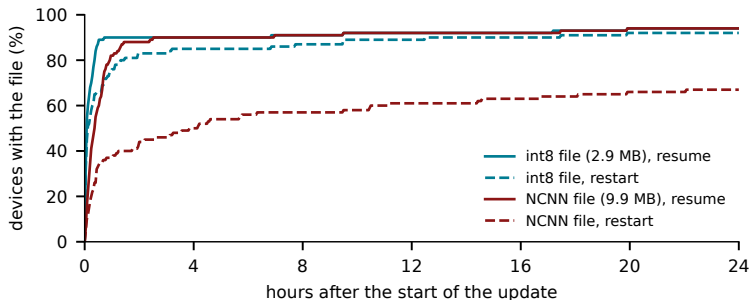
Source: Formula: *Machine Learning Systems*, Volume II, chapter “ML Operations at Scale”.

File, download	50 %	90 %	No file after 7 days	Bytes sent / byte
int8 (2.9 MB), restart	7 min	12.5 h	1	7.1
int8 (2.9 MB), resume	4 min	42 min	0	1.0
NCNN (9.9 MB), restart	3.8 h	never	17	60.7
NCNN (9.9 MB), resume	21 min	2.5 h	0	1.0

- ▶ 100 devices. Link speed: median 0.21 Mbit/s. The link is up for 2 minutes and down for 3 minutes on average (up 39 percent of the time). 10 devices are off and come back after 1 to 72 hours.
- ▶ “Restart”: a lost link starts the download again from byte 0. “Resume”: it continues from the last byte (HTTP Range).

Source: File sizes of the Day 8 lab.

How fast the fleet gets the file



- ▶ With a restart, a slow device whose link drops before the end never finishes the large file. It sends many bytes for nothing.
- ▶ With a resume, all devices that are on get both files. The last devices are the ones that were off: the time of the full fleet does not depend on the file.
- ▶ Small files help twice: a shorter download, and less chance of a lost link during it. The int8 file has 29 percent of the size of the NCNN file.

Source: Simulation of this part.

A staged rollout in the same simulation

Stage	Devices	Start	90 % have it	All have it
Canary	5	0 h	0.7 h	0.7 h
Stage 2	25	24.7 h	27.4 h	69.9 h
Stage 3	70	51.4 h	51.8 h	57.9 h

- ▶ The int8 file with resume. The next stage starts 24 hours after 90 percent of a stage have the file: the health check.
- ▶ The canary takes devices that are on. A canary that waits for a device that is off waits for days.
- ▶ 90 percent of the fleet run the new version after 51.8 hours, in place of 0.7 hours with no stages. A bad version reaches at most 5 devices in the first day.

Source: Simulation of this part.

Device class	Artifact	Update	Test
XIAO ESP32S3	Program with an <code>int8</code> model (TFLM)	A/B program slots	Self-test on the board
Raspberry Pi 5	LiteRT <code>int8</code> or NCNN folder	Model folder and link	Latency and accuracy on the board
Older Raspberry Pi	A smaller model, a smaller image size	The same	Its own latency budget

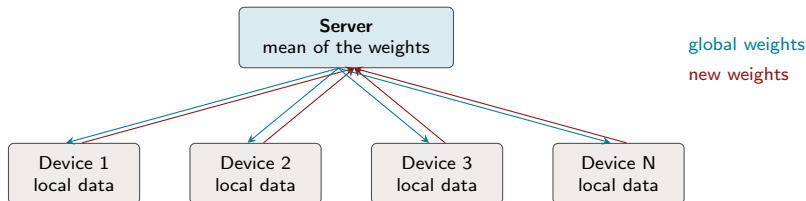
- ▶ One trained model gives several artifacts. Each device reports its class, and the server sends the artifact of that class.
- ▶ Test each artifact on a real device of its class. An accuracy test on a laptop does not show that a model is too large or too hot for the board.
- ▶ Devices that are often off keep old versions for months. The server must accept the messages of old versions (version skew).

Source: *Machine Learning Systems*, Volume II, chapter “ML Operations at Scale” (edge fleet controls).

What changes	Cost on the device	Example
Thresholds, a calibration	Almost none	Day 13: the reference of each camera
The biases only	Little memory, a few values	Microcontroller class
Small adapter layers	Moderate memory, the backbone stays fixed	Phone class
Selected layers	Activations and gradients in memory, much energy	Large devices only

- ▶ Training needs memory for the activations and the gradients, not only for the weights. A device that runs a model cannot always train it.
- ▶ Each device then has its own model state. The versions now include the local state of each device.
- ▶ Adapt only when the device is idle, has power, and stays below its temperature limit.

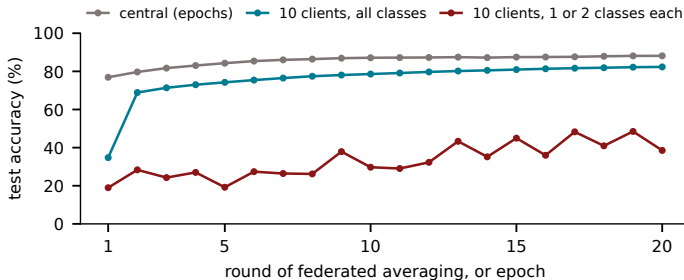
Source: *Machine Learning Systems*, Volume II, chapter “Edge Intelligence” (the adaptation of each device class).



- ▶ Each round: the server sends the model; each device trains on its own data; the server takes the mean of the new weights, weighted by the number of samples (federated averaging).
- ▶ The raw data stays on the device. Only weights cross the network.
- ▶ Not private by itself: weights and gradients can leak facts about the data. Secure aggregation and differential privacy add protection.

Source: *Machine Learning Systems*, Volume II, chapter "Edge Intelligence". Federated averaging: McMahan et al., 2017.

Measured: federated averaging



- ▶ The Day 6 student CNN (6218 parameters), Fashion-MNIST, 10 clients, 1 local epoch in each round.
- ▶ After 20 rounds: 82.3 percent with all classes on each client; 36.0 to 48.5 percent in the last 5 rounds with 1 or 2 classes. Central training: 88.2 percent after 20 epochs.
- ▶ Data that differs from device to device (non-IID) makes the local models pull in different directions.

Source: Experiment of this part (Keras, x86 processor, seed 0).

One update of the model (6218 values in float32)	24.3 KB
One round: 10 downloads and 10 uploads	485.8 KB
20 rounds	9.49 MB
The raw images of one client (6000 images of 28×28)	4.49 MB

- ▶ For this small model, one update is much smaller than the data of one client. For a model of millions of parameters, the rounds add up to more than the data.
- ▶ Each round needs devices that are on, have power, and have a link. Slow devices (stragglers) must not stop a round.
- ▶ Use it when the data must stay on the devices and the devices can train. Else: collect a sample of data (Day 13) and train centrally.

Source: Calculation with the model of the experiment; 1 KB = 1024 bytes.

Your company has 100 Raspberry Pi 5 cameras with the Day 8 detector. They are in shops with a bad mobile link: about 0.2 Mbit/s, and the link drops every few minutes. 10 shops switch their devices off at the weekend. You have a new model.

- ▶ Which file do you send, and how is it downloaded?
- ▶ What does a device check before and after the switch?
- ▶ In which stages do you roll out, and how long does each stage wait?
- ▶ When do you roll back, and who decides?

- ▶ **File:** the int8 file (2.9 MB, not 9.9 MB) with a manifest: version, size, SHA-256, signature, oldest program. The download resumes after a lost link.
- ▶ **On the device:** check the hash and the signature; put the file in a new version folder; switch the link; restart; run a self-test on stored frames. Keep the old folder.
- ▶ **Stages:** 5 canary devices that are on (one of each shop type), then 25, then 70. Each stage waits 24 hours of health checks. In the simulation: 90 percent after 51.8 hours.
- ▶ **Roll back:** the device itself when the self-test fails or the program crashes again and again; the fleet server when the health check fails; a person when the model metrics of the canary change against the old version. Stop the rollout at once.

Part 1 Life cycle in production

Exercise: write the update plan for 100 devices with a bad network

Part 2 Security, privacy, and reliability

Exercise: list the threats for the Day 12 system and select two mitigations

Part 3 Design method

Exercise: review a given system design and find three weak points

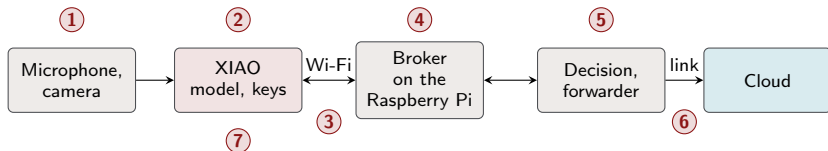
- ▶ A device stands in a public place for years. A person can touch it, open it, or take it away.
- ▶ It has a network link, a model, keys, and data of the people near it. Each one is worth something to an attacker.
- ▶ Many identical devices share one weakness. One method attacks all of them.
- ▶ Example: in 2016 the Mirai botnet took control of more than 600 000 cameras, recorders, and routers with factory-default passwords, and used them for an attack of 1.2 Tbit/s.

Three goals

Confidentiality: only the right people read the data and the model. **Integrity:** nobody changes the data, the model, or the commands. **Availability:** the device works when it is needed.

Source: *Machine Learning Systems*, Volume II, chapter “Security & Privacy” (Mirai).

The attack surface of the Day 12 system



- | | |
|---|---|
| 1 A false input: a sound, a photo, a light | 5 The SD card: the queue, the logs, the model files |
| 2 The flash: the program, the model, the Wi-Fi password | 6 The link to the cloud |
| 3 The radio: read or send messages | 7 The USB and debug port, the update channel |
| 4 The broker: any client can connect | |

The attack surface is each point where an attacker can read, change, or stop something. Each point needs its own defense.

Threat	Example	It harms
Eavesdropping	Read the MQTT messages on the Wi-Fi	Privacy
False commands	Publish led=1 to a device	Integrity, safety
Model theft	Copy the model file from the SD card	The value of the model
Model extraction	Query the model and train a copy	The value of the model
False input	A sound or a photo that fools the model	The decisions
Poisoned data	False samples for the next training	The next model
Physical access	Read the flash, use the debug port	All of the above
Denial of service	Flood the broker, jam the Wi-Fi	Availability

Select the defenses by risk: how likely is the threat, and how bad is the result for this system?

Source: After *Machine Learning Systems*, Volume II, chapter “Security & Privacy”.

Test	Open	Passwords, rule	TLS too
Messages of g07 that a stranger reads	1	0	0
False led=1 that the g07 device gets	1	0	0
Client with no password	accepted	refused	refused
Client with a wrong password	accepted	refused	refused
TLS client that does not know the CA			refused

- ▶ The lab broker of Day 12 accepts any client. A stranger in the Wi-Fi subscribes to #, reads all messages, and switches the LED of another group.
- ▶ In the second and third broker the stranger is the user g08: a valid user of a different group.
- ▶ The lab network is closed, so the lab accepts this risk. A product cannot.

Source: Experiment of this part: three Mosquitto 2.0.11 brokers on one computer.

```
mosquitto_passwd -c /etc/mosquitto/passwd g07  # asks for the password of g07
```

```
# /etc/mosquitto/conf.d/lab.conf
listener 1883
allow_anonymous false
password_file /etc/mosquitto/passwd
acl_file /etc/mosquitto/acl

# /etc/mosquitto/acl: each user only in the tree of its own group
pattern readwrite edgeai/%u/#
```

- ▶ %u is the user name. The user g07 reads and writes only below edgeai/g07/.
- ▶ A message to a topic that the rule denies is dropped with no error, and a subscription to # gets nothing of the other groups.
- ▶ Better: one user for each device, so that a stolen device can be removed alone.

Source: The settings of the experiment of this part; the file names are those of a Raspberry Pi.

	Plain	TLS
Connect, median	2.4 ms	51.9 ms
Bytes to connect	36	3368
Bytes of 100 messages	10 400	12 600
One message	104 bytes	126 bytes

An IMU message of 81 bytes of JSON, QoS 0, with a password. A counting proxy measured the bytes.

- ▶ TLS encrypts each message and checks that nobody changed it. The certificate proves the identity of the broker.
- ▶ The cost is at the start: a handshake with certificates. Keep the connection open: each message then costs 22 bytes more.
- ▶ The ESP32-S3 has AES, SHA, and RSA hardware that makes TLS possible on the XIAO (port 8883 by convention).

Source: Experiment of this part (OpenSSL 3.0.2, x86). ESP32-S3 data sheet 2.2, section "Security".

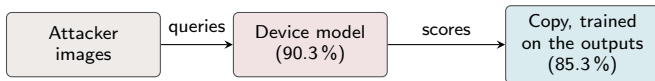
- ▶ **One key for each device.** A shared password lets one stolen device open all others. A key for each device can be revoked alone.
- ▶ **No default password.** The Mirai botnet used factory passwords. A device sets its key at installation.
- ▶ **Not in the code repository.** The Day 12 sketch keeps the Wi-Fi password in `arduino_secrets.h`, outside the shared code. A key in git stays in its history.
- ▶ **Protected on the device.** The ESP32-S3 has 4096 bits of eFuse memory, an HMAC unit, and an RSA signature unit whose private key only the hardware can decrypt. On a Raspberry Pi, a key is a file: set its rights (`chmod 600`).
- ▶ **Change them.** Plan how to replace a key or a certificate before it expires.

Source: ESP32-S3 data sheet 2.2, sections “Security”, “HMAC Accelerator”, “RSA Digital Signature Peripheral”.

```
interp = tf.lite.Interpreter("mnv2_int8.tflite")    # the Day 7 lab file
interp.allocate_tensors()
# 109 constant tensors with 3 487 830 values, for example:
# shape [1280, 1, 1, 320]: -17, 29, -1, -44, 80, 55, 62, 23, ...
```

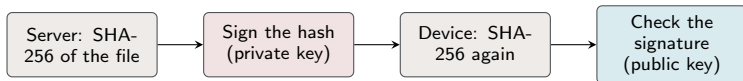
- ▶ A person with the SD card or the flash of a device has the model: the standard interpreter reads each weight.
- ▶ Encryption at rest: AES-256-GCM of the 4 107 968-byte file took 13.1 ms, its decryption 13.1 ms (one x86 core). A changed byte makes the decryption fail.
- ▶ The weak point is the key: it must be on the device to decrypt. The ESP32-S3 has flash encryption in hardware. A person who takes the SD card of a Raspberry Pi reads all files that the software did not encrypt.

Source: Experiment of this part (LiteRT interpreter of TensorFlow, package cryptography).
ESP32-S3 data sheet 2.2.



- ▶ Day 6: a small student trained only on the scores of the teacher for 60 000 images with no label reached 85.3 percent. The teacher had 90.3 percent. This is a copy through the outputs.
- ▶ An attacker needs no file, only many queries and the scores.
- ▶ Defenses: give only the class, not all scores; limit the queries of each client; watch for many unusual queries; mark the model (watermark) to prove a copy.

Source: Day 6, Part 2 (distillation experiment). Defenses: *Machine Learning Systems*, Volume II, chapter "Security & Privacy".



- ▶ A hash proves that the file did not change in transit. A signature also proves who made it: only the holder of the private key can sign.
- ▶ Cost on one x86 core: SHA-256 of the 4 107 968-byte file 11.3 ms; an Ed25519 signature of 64 bytes, its check 0.14 ms.
- ▶ Secure boot applies the same check to the program at each start. The ESP32-S3 boots only firmware with a valid RSA-PSS signature when secure boot is on.

Source: Experiment of this part. ESP32-S3 data sheet 2.2, section “Secure Boot”.

Attack	Defense
Read the SD card or the flash	Encrypt the data and the model; keep no data that the task does not need
Use the USB or the debug port	Disable the debug port (an eFuse of the ESP32-S3); no login with a default password
Load a different program	Secure boot; signed updates
Replace or cover the sensor	Input checks (Day 13); a second sensor; a tamper switch
Take the device away	A key for each device that the server can revoke

- ▶ With physical access, a network password does not help. The defense must be in the hardware and in the boot chain.
- ▶ A case with a lock and a mount out of reach are also defenses.

Source: *Machine Learning Systems*, Volume II, chapter “Security & Privacy” (physical attacks).
ESP32-S3 data sheet 2.2.

- ▶ A model has no common sense. A printed photo, a sticker, a sound from a speaker, or a light can change its output.
- ▶ On a device, the attacker controls the scene, not the code: the attack goes through the sensor.
- ▶ Defenses that this course built:
 - input checks: brightness, sharpness, value range (Day 13);
 - the decision layer: several windows, hysteresis, a second condition before an action (Day 12);
 - a second sensor that must agree for an important action.
- ▶ Poisoned data: a sample that an attacker places in the next training set. Check the origin of new data before you train (Day 13, the loop back to training).

Key point: Do not let one model output start an action that can hurt somebody.

Rule	In the course systems
Process on the device	The XIAO sends a keyword event, not the audio (Day 12)
Send results	A count each minute, not the video (Day 12, Part 3)
Minimize the logs	Counts and scores, no images (Day 13)
Limit the time	Rotation, a retention for each folder (Day 13)
Protect the link	Passwords, access rules, TLS (this part)

- ▶ Data that never leaves the device cannot leak from the cloud.
- ▶ Federated learning keeps the raw data local, but the weights can still leak facts (Part 1).
- ▶ Tell the people near the device what it records.

```
[Service]                                # a unit of systemd for the detector
Type=notify
ExecStart=/home/edge/yolo/bin/python monitor_detect.py ...
WatchdogSec=3                            # no ping for 3 s: stop and restart
Restart=on-failure
RestartSec=1
```

- ▶ The program sends WATCHDOG=1 to systemd in each loop. A program that hangs stops sending, and systemd restarts it.
- ▶ Test with a program that stopped its pings after 6 s: systemd started it again 4.3 s and 4.5 s after the last ping.
- ▶ The ESP32-S3 has three watchdog timers in hardware. Two of them are on during the boot, to recover from a failed start.

Source: Experiment of this part (systemd-run --user with the same settings). ESP32-S3 data sheet 2.2, section “Watchdog Timers”.

Failure	Reduced operation	Report
Camera read fails	No decisions from the camera; a safe state	ERROR, a count
New model fails its test	The old model (Part 1)	The version that runs
Board too hot	A smaller model or fewer frames	Temperature, throttled bits
Link to the cloud lost	Local decisions, a queue (Day 12)	Queue length
Disk almost full	Stop the detailed log	Free disk
Clock not set	Mark the times as not trusted	A flag in each message

- ▶ A device that does less, and says so, is better than a device that stops or does wrong things in silence.
- ▶ Each reduced mode needs a test: cut the camera, fill the disk, stop the link, as in the Day 12 lab.

The lab systems against a product

Topic	The labs	A product
Broker	Open, no TLS	A user for each device, access rules, TLS
Keys	One Wi-Fi password in a file	A key for each device, in hardware where possible
Updates	Upload by cable	Signed OTA updates with rollback (Part 1)
Model file	Plain file	Encrypted, signed, versioned
Restart	By hand	Watchdog, reduced modes
Data	Logs on the card	Minimized, rotated, retention

The labs keep the setup short. Your capstone design names which rows of the right column it needs, and why.

The Day 12 system: the XIAO sends keyword events with MQTT to the broker on the Raspberry Pi; `decide.py` switches the LED of the XIAO; the forwarder sends the decisions to the cloud broker. The lab broker is open and has no TLS.

1. List at least five threats. For each one, name the point of the attack surface and the goal that it harms.
2. Select two mitigations that remove the largest risk for the least work. Say what each one costs.

- ▶ **False commands:** any client publishes `led=1` (broker; integrity). **False events:** any client publishes “yes” (broker; integrity). **Eavesdropping** on the Wi-Fi (radio; confidentiality). **Flood** of the broker (broker; availability). **Wi-Fi password** read from a stolen XIAO (flash; confidentiality). **Queue and logs** read from the SD card (storage; privacy). **A sound** that the model takes as “yes” (sensor; integrity).
- ▶ **Mitigation 1:** a user for each device and the access rule `pattern readwrite edgeai/%u/#`. It stops the false commands and events of the measured test. Cost: a password file and one setting on each device.
- ▶ **Mitigation 2:** TLS on the broker. It stops the eavesdropping. Cost in the test: 51.9 ms to connect and 22 bytes more for each message; certificates to manage.
- ▶ Other good answers: flash encryption on the XIAO, a rate limit, a second condition in `decide.py`.

Part 1 Life cycle in production

Exercise: write the update plan for 100 devices with a bad network

Part 2 Security, privacy, and reliability

Exercise: list the threats for the Day 12 system and select two mitigations

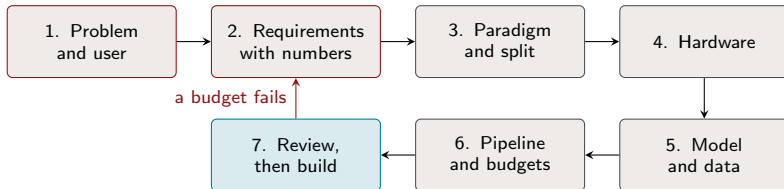
Part 3 Design method

Exercise: review a given system design and find three weak points

- ▶ On Days 1 to 13 each lab gave you the task, the boards, and the model. In a project you select them.
- ▶ A change on paper costs minutes. The same change after the build costs days. After the installation of 100 devices, it needs an update of each device (Part 1).
- ▶ Most failures of a deployed system come from a requirement that nobody wrote down: the battery life, the light at night, the network of the site.
- ▶ A design gives the team one plan with numbers. A review finds the weak points before the build.

This part

A method in seven steps, applied to one example. Then the budgets for power and cost, the design review, and the capstone project of Days 14 and 15.



- ▶ Each step uses the result of the step before it. The requirements select the paradigm, the paradigm limits the hardware, and the hardware limits the model.
- ▶ When a budget fails, go back: change the requirement, the hardware, or the model. Do not hope that the build solves it.
- ▶ The steps follow the prototype process of the companion book: the problem, the needs, the hardware, the software, the test.

Four questions

- ▶ **For whom?** The user and the people near the device.
 - ▶ **Where?** The place, the power, the network, the light, the temperature.
 - ▶ **What?** The function that solves the problem.
 - ▶ **Which benefit?** What is better than today?
-
- ▶ Write the problem, not a solution: “detect a failing bearing”, not “run YOLO on a camera”.
 - ▶ Start with a minimum viable product: the smallest system that shows that the idea works.

Example of this part

A workshop has 20 machines. A bearing that fails makes the machine vibrate. Today a technician hears it, often too late. The owner wants an alarm at the machine and a dashboard for the technician.

Step 2: requirements with numbers

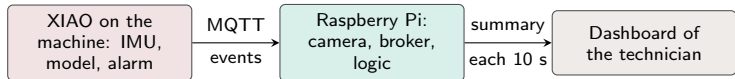
Vague	With a number	Test
Fast	Alarm within 3 s of the start of the vibration, in 95 percent of the events	Timed events in the lab
Accurate	Recall of 0.95 for “abnormal” on recordings of the site	Test set of the site
Small	Model and program fit 3 MB of flash and 300 KB of RAM	Build output, arena size
Long battery	30 days on one battery	Power budget, then a meter
Private	No image leaves the device	Read the MQTT topics
Robust	Local alarm with no network for 24 h	Cut the link
Cheap	Hardware below a fixed price for each site	Cost budget

A requirement with no number cannot fail a test, so nobody can check it. Each requirement needs a test that you can run before the demonstration.

Example: a machine monitor

- R1 Classify the vibration of each machine as normal or abnormal.
 - R2 Alarm at the machine within 3 s of the start of abnormal vibration.
 - R3 Alarm with no network; the dashboard can wait.
 - R4 Warn when a person is in the zone of a machine with an alarm.
 - R5 Dashboard for the technician: state of each machine, each 10 s.
 - R6 Sensor on a moving part: no cable, if possible.
 - R7 No video leaves the workshop.
 - R8 Model updates for all machines from one place (Part 1).
- ▶ R2, R3, and R7 decide where the computation runs.
 - ▶ R6 is a wish that has a cost: the next frames calculate it.

Step 3: paradigm and split of the work



Apply the filters of Day 1 to each requirement:

- ▶ R2 and R3: the alarm cannot wait for a network. The decision runs on the XIAO at the machine (TinyML).
- ▶ R4 and R7: a camera model needs more than a microcontroller, and the video must stay local. A Raspberry Pi in the workshop runs it (edge ML).
- ▶ R5 and R8: a broker, a log, and the updates need a computer that is always on. The Raspberry Pi again.
- ▶ No requirement needs the cloud. A cloud copy of the summary is an option.

	XIAOML Kit	Raspberry Pi 5 (8 GB)
Memory	512 KB RAM, 8 MB PSRAM, 8 MB flash	8 GB RAM, microSD card
Sensors	IMU, microphone, camera	Camera on a cable, USB
Power	Chip 217 mW at work; Sense board 19 mW in light sleep	5 to 7 W at a moderate load
Runs well	Small models in <code>int8</code> (Days 3 to 5)	Detection, language models, Python (Days 7 to 13)
Price	about \$40	about \$80

- ▶ Criteria: can it do the task, the power, the size, the cost, the life, the tools that the team knows.
- ▶ Select the smallest board that meets the requirements with a margin. A larger board costs power and money at each site.

Source: Day 1 (memory); Day 9 (power: ESP32-S3 data sheet 2.2, Seeed, *Edge AI Engineering*); prices: textbook kit pages (2025).

Step 5: a model that fits the budget

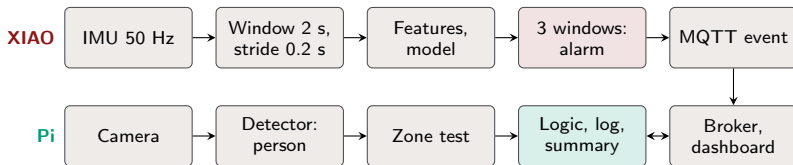
Task	Model	Value	Board
Motion	Features, dense 63, 20, 10, 4 (1534 parameters)	int8 file 4960 bytes, arena 1028 bytes	XIAO
Keyword	2 Conv1D, 1 dense (1556 parameters)	49 × 13 features each 0.5 s	XIAO
Image	MobileNetV2 0.35, 96 pixels, int8	Peak 138 240 bytes of RAM	XIAO
Detection	YOLO11n, 320 pixels, NCNN	About 80 ms for each image	Pi 5
Language	llama3.2:1b, Q8_0	1.32 GB, 8.99 tokens/s	Pi 5

- ▶ Start from a model of the course with known numbers. Check the three budgets of Day 1: flash, RAM, and operations.
- ▶ Plan the data: who records it, where, how many sessions, how you label it (Day 2). For R1, record each machine normal and with a known fault.

Source: Days 3, 4, 5, 8, 10 of this course (arena: 32-bit build on the work computer).

Pi 5 times: *Edge AI Engineering*, textbook kit lab.

Step 6: the pipeline of the example



- ▶ Each box is one step that you can test alone, with a recorded input (Day 12: test the decision layer).
- ▶ The alarm path is on the XIAO only. The Pi adds the person test, the log, and the dashboard.
- ▶ Write the message format and the topics before the build: `edgeai/gNN/<device>/<channel>` (Day 12).

Stage	Budget
Fill the window	2.0 s
Features and model	0.1 s
Two more windows (stride 0.2 s)	0.4 s
Event to the Pi (MQTT)	0.1 s
Total	2.6 s
Requirement R2	3.0 s
Margin	0.4 s

- ▶ The window takes most of the budget, not the model.
- ▶ A shorter window is faster, but it can lose accuracy: test both on the data.
- ▶ The alarm on the XIAO comes before the MQTT step. The dashboard is not in the path.

Give each stage a budget, then measure each stage in the lab (Day 9) and compare.

Source: Budget values of this example. Window and stride: Day 2. The lab measures the real values.

Design (battery 1000 mAh, 3.7 V: 3700 mWh)	Mean power	Life
Wi-Fi always connected (Sense board in modem sleep)	at least 167.2 mW	at most 22 h
Model 50 ms for each 2 s window, light sleep between, Wi-Fi 3 s each hour	24.2 mW	6.4 days

- ▶ Mean power: $P = d \cdot P_{\text{work}} + (1 - d) \cdot P_{\text{rest}}$ with the duty cycle $d = 50/2000 = 2.5$ percent (Day 9).
- ▶ In the second design, the light sleep of the Sense board (19 mW) is 77 percent of the energy. The model is not the problem.
- ▶ 30 days need 17.4 Wh: a battery of about 4700 mAh. Or change R6: a cable from a USB supply at the machine.

Source: Day 9 values (ESP32-S3 data sheet 2.2; Seed board values).

Power budget: the Raspberry Pi on mains power

	5 W	7 W
Energy each day	120 Wh	168 Wh
Energy each year	43.8 kWh	61.3 kWh
100 sites, each year	4380 kWh	6132 kWh
Cost at 0.10 USD for each kWh	438 USD	613 USD
Power bank of 10 000 mAh at 3.7 V (37 Wh)	7.4 h	5.3 h

- ▶ A Raspberry Pi 5 at work needs a power supply at the site. A battery gives hours, not days.
- ▶ The power supply must give the peak: the course uses a 27 W supply. A weak supply gives the undervoltage flag of Day 9.
- ▶ Heat: the board throttles above 80 °C. A closed box at a warm machine needs the active cooler and air flow.

Source: 5 to 7 W at a moderate load: *Edge AI Engineering*, chapter “Setup”.

One site: one machine, its own Pi	USD
XIAOML Kit	40
Raspberry Pi 5, 8 GB	80
Camera Module 3	25
Other parts: power supplies, cooler, card, case, cables	40
One site	185
100 sites and 15 percent of spares: 115 sets	21 275

- ▶ The Pi is 43 percent of one site. The workshop: 20 kits and one Pi with one camera cost 945 USD, 20 sites cost 3700 USD. The split of the work changes the cost.
- ▶ The hardware is often the smallest part of the cost. The next frame gives the running costs.
- ▶ Prices change, the price of memory most. Ask the reseller on the day of the order.

Source: Kit pages of the textbook (2025); raspberrypi.com, "Camera Module 3" (2026-10-03).

Data of one site in 30 days	Volume
Video, H.264 1280 × 720 (2.62 Mbit/s)	848.4 GB
Raw IMU at 50 Hz, one JSON message each (147 bytes)	19.1 GB
Detections, 10 each second	4.3 GB
A count each minute	5.1 MB

- ▶ Running costs: energy, the network, a cloud service, the replacement of failed devices, and the time of the people who watch and update the system.
- ▶ One model update (2.9 MB, with resume) for 100 sites: 302 MB. Four each year: 1.2 GB. Small against the video.
- ▶ The data that leaves the site sets the cost of the link and of the cloud: send results, not raw data (Day 12).

Source: Day 12, Part 3 (data rates); Day 14, Part 1 (update). Here 1 GB = 10^9 bytes.

Compare the options

	XIAO only	Pi only	XIAO and Pi	XIAO to cloud
R2: alarm within 3 s	✓	✓	✓	network
R3: alarm with no network	✓	✓	✓	✗
R4: person in the zone	✗	✓	✓	✗
R5: dashboard	✗	✓	✓	✓
R6: no cable on the machine	battery	✗	battery	battery
R7: no video leaves	✓	✓	✓	✓
Data to the cloud in 30 days	none	none	none	19.1 GB

- ▶ “Pi only” needs a sensor cable from each machine to the Pi.
- ▶ “XIAO and Pi” meets R1 to R5, R7, and R8. R6 needs a battery of about 4700 mAh, or a cable: the team decides with the owner.

Key point: Write the options and the reason for your choice. The review checks the reason.

Test the riskiest assumption first

Assumption	First test	Plan B
The IMU separates a failing bearing	Record one machine with a known fault	A microphone; a threshold on the RMS
The Wi-Fi reaches each machine	Read the signal strength at each machine	A cable; a second access point
The camera sees the zone in bad light	Test images at night (Day 13)	More light; a second camera
The battery lasts	Measure the mean current (Day 9)	A larger battery; a cable

- ▶ Build the riskiest part first, as a minimum viable product. If it fails, the project changes early.
- ▶ A part that the labs already showed (MQTT, the dashboard) has a low risk: build it later.

1. Each requirement has a number and a test.
2. Each budget has a value and a source: latency, memory, power, cost. A guess is marked as a guess.
3. The model fits the board: flash, RAM, time.
4. The data: where it comes from, how many samples, who labels it, and a test set that the training does not see.
5. The failure modes: no network, a failed sensor, a bad update (Parts 1 and 2).
6. The threats and the data that leaves the device (Part 2).
7. The riskiest assumption and its first test.

The reviewer asks questions; the team answers with numbers. A weak point is a result of the review, not a failure of the team.

Weak points that this course measured

Weak point	What the course measured	Day
Accuracy on training images	mAP50 0.670 on 128 training images, 0.546 on 5000 test images	8
Calibration with one class	69.2 percent in place of 100.0	4
Radio always on	At most 22 h on 1000 mAh	9, 14
An open broker	A stranger read 1 of 1 messages and switched an LED	14
QoS 0 and a lost link	51 messages lost in a cut of 1.5 s	12
Dark images	38.4 percent in place of 77.4 percent	13
A list that grows	65.6 MB each hour	13

A reviewer who knows these numbers asks the right questions. Use them in your own review.

Source: Results of Days 4, 8, 9, 12, 13, and 14 of this course.

Your system must:

1. Use the XIAOML Kit and the Raspberry Pi.
2. Run at least one optimized model, with the size and the accuracy before and after the optimization.
3. Make one decision locally, and continue to work without the internet.
4. Send telemetry with MQTT to a dashboard.
5. Include a benchmark table with latency, memory, and energy.

Rubric (Day 15)	Weight
The system works in the demonstration	30 percent
Integration of the two boards, messages, and monitoring	25 percent
Measured results and analysis of trade-offs	25 percent
Report and presentation	20 percent

The proposal: one page

- ▶ The problem and the user
- ▶ The boards and the split of the work
- ▶ The model and its optimization
- ▶ The data: source, size, labels
- ▶ The metrics and the budgets, with numbers
- ▶ How it meets the five requirements

When	What
Day 14, A	Teams (20 min)
Day 14, B	Proposal (50 min)
Day 14, C	Design review (40 min)
Day 14, D	Build (40 min)
Day 15, A	Build, measure (80 min)
Day 15, B	Demonstration (10 min)
Day 15, C	Report, feedback

Teams of three or four students. Keep the scope small: reuse the labs and add one new part. A small system that works scores more than a large system that does not.

A team proposes this design:

- ▶ A XIAOMI Kit on a pole at the parking entrance, on a battery of 1000 mAh. Wi-Fi stays connected.
- ▶ Each second, it sends a camera image of 96×96 pixels to a cloud server. YOLO11n at 640 pixels detects cars there.
- ▶ The cloud sends “open” to the gate when it detects a car.
- ▶ Accuracy: 99 percent on the training images.
- ▶ The team updates the model with a USB cable. The broker has no password.

Find three weak points. For each one, give a number or a result of the course, and propose a change.

- ▶ **Power:** the Sense board alone needs at least 167.2 mW with Wi-Fi connected: at most 22 h on 1000 mAh, before the camera. Change: mains power, or a local model and light sleep (Day 9).
- ▶ **No internet, no gate:** the decision is in the cloud, so the gate fails with the link (requirement 3). Change: detect on the device, send only events (Day 12).
- ▶ **Accuracy on training images:** Day 8 gave 0.670 on training images and 0.546 on test images. Change: a test set of the site.
- ▶ **Model and images:** YOLO11n at 640 pixels on an image of 96 pixels gives no detail; the images can show persons and number plates (privacy, Part 2).
- ▶ **Updates and broker:** a USB cable on a pole does not scale (Part 1); an open broker accepts a false “open” (Part 2).

- ▶ Each device knows the version of its model and its program. An update goes to a few devices first, then to more. A device keeps the old version and returns to it when the new one fails.
- ▶ The attack surface of an edge device includes its network, its storage, and its physical access. Encrypted transport, a key for each device, signed updates, and data minimization reduce the risks.
- ▶ A watchdog restarts a stopped program. A device with a failed part goes to a safe, reduced operation and reports it.
- ▶ A design starts from the requirements: the latency, the energy, the privacy, and the cost. They select the paradigm, the boards, the model, and the pipeline. A review finds the weak points before the build.

Key point: Plan the updates, the threats, and the budgets before the build, because a deployed device is hard to change.

Book chapters

- ▶ *Machine Learning Systems* (mlsysbook.ai), Volume II, chapter “Edge Intelligence”, sections “Design Constraints” and “Production Integration”.
- ▶ *Machine Learning Systems*, Volume II, chapter “Security & Privacy”.
- ▶ *XIAO: Big Power, Small Board*, chapter 2.1: “Introduction to Product Prototype Design”.

Papers

- ▶ “Communication-Efficient Learning of Deep Networks from Decentralized Data”, McMahan et al., 2017.
- ▶ “Stealing Machine Learning Models via Prediction APIs”, Tramèr et al., 2016.

Tools

- ▶ The Update Framework: <https://theupdateframework.io/>
- ▶ Mender (over-the-air updates): <https://docs.mender.io/>

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *Edge AI Engineering: Raspberry Pi* by Marcelo Rovai.
github.com/Mjrovai/EdgeML_Made_Ease_ebook and
github.com/Mjrovai/EdgeML-with-Raspberry-Pi, GPL-3.0.
- ▶ *XIAO: Big Power, Small Board* by Lei Feng and Marcelo Rovai.
github.com/Mjrovai/XIAO_Big_Power_Small_Board-ebook, GPL-3.0.